

Documentação do Projeto de Extensão — Doces com Amor

Grupo 9 — Pesquisa e Inovação

Integrantes: Gabriela Vilegas, Guilherme Santos, João Carmo e Nicolas Javed

Instituição: São Paulo Tech School

Ano: 2026

Sumário

1. Contexto
2. Objetivos
3. Justificativa
4. Escopo
5. Requisitos
6. Premissas
7. Restrições
8. Artefatos relacionados às regras de negócio
9. Artefatos relacionados às arquiteturas
10. Product backlog
11. Ferramentas de planejamento e organização
12. Links do projeto no Git
13. Referências bibliográficas e técnicas

Visão rápida

O Doces com Amor é um sistema interno para organizar os pedidos de uma doceria. A ideia é deixar em um só lugar as informações que antes podiam ficar espalhadas em mensagens, anotações e agendas. O sistema permite cadastrar clientes, endereços e pedidos, acompanhar datas pelo calendário e consultar informações de pagamento e entrega.

O projeto foi desenvolvido com React no frontend, Java com Spring Boot no backend e MySQL no banco de dados. Também possui integração com Google Calendar, containers Docker e infraestrutura AWS criada com Terraform.

1. Contexto

A Doces com Amor trabalha com pedidos que precisam ser preparados e entregues em datas e horários combinados. Quando esse controle é feito de forma manual, fica mais fácil esquecer uma entrega, perder uma informação do cliente ou não ter clareza sobre o pagamento.

O projeto surgiu para apoiar a rotina da proprietária e da equipe da doceria. Ele não é uma loja virtual para o cliente final. É uma ferramenta de uso interno para consultar o que precisa ser feito e manter os pedidos organizados.

Os principais problemas trabalhados foram:

- informações de pedidos espalhadas em lugares diferentes;
- dificuldade para visualizar os próximos pedidos;
- risco de esquecer datas, horários e cobranças;
- falta de um histórico centralizado de clientes e endereços;
- pouca visão sobre pedidos cancelados, reagendados e concluídos.

2. Objetivos

Objetivo geral

Desenvolver um sistema web que ajude a Doces com Amor a cadastrar, consultar e acompanhar seus pedidos de forma simples.

Objetivos específicos

- centralizar clientes, endereços e pedidos em um banco de dados;
- permitir a consulta dos pedidos por data em um calendário;
- mostrar os próximos pedidos em ordem de entrega;
- acompanhar o status de pagamento e de entrega;
- permitir edição, cancelamento e reagendamento de pedidos;
- apresentar indicadores básicos em um dashboard;
- registrar os pedidos na agenda do Google quando a integração estiver ativa;
- disponibilizar o sistema em uma infraestrutura AWS reproduzível.

3. Justificativa

A organização dos pedidos interfere diretamente na produção e no atendimento. Um horário esquecido ou um pagamento sem acompanhamento pode gerar prejuízo e desgaste com o cliente.

A solução proposta é pequena o suficiente para ser usada no dia a dia, mas já resolve pontos importantes: reúne os dados, facilita a consulta por data e reduz a dependência de anotações manuais. Para o grupo, o projeto também permitiu aplicar conhecimentos de frontend, backend, banco de dados, cloud e trabalho em equipe em um problema real.

4. Escopo

Incluído nesta versão

- login administrativo e troca obrigatória da senha inicial;
- cadastro, consulta, edição, ativação e inativação de clientes;

- cadastro, consulta, edição e exclusão de endereços;
- cadastro e listagem de pedidos;
- calendário para filtrar pedidos por data;
- consulta dos próximos pedidos;
- detalhes, edição, cancelamento e reagendamento de pedidos;
- atualização dos status de pagamento e entrega;
- relatórios e dashboard;
- integração com Google Calendar;
- API REST documentada com Swagger;
- containers Docker para frontend e backend;
- publicação das imagens no Amazon ECR pelo GitHub Actions;
- infraestrutura AWS criada com Terraform;
- métricas e alarmes básicos no CloudWatch;
- simulação local dos buckets Bronze, Silver e Gold com LocalStack.

Fora do escopo desta versão

- aplicativo para o cliente fazer o próprio pedido;
- pagamento online dentro do sistema;
- integração com aplicativos de entrega;
- emissão de nota fiscal;
- controle de estoque e compra de ingredientes;
- aplicativo mobile nativo;
- funcionamento offline;
- Data Lake integrado de ponta a ponta;
- suporte e manutenção após o encerramento do projeto acadêmico.

5. Requisitos

Requisitos funcionais

ID	Requisito
RF01	O administrador deve conseguir entrar com usuário e senha.
RF02	No primeiro acesso, o administrador deve trocar a senha inicial.
RF03	O sistema deve permitir cadastrar, consultar e editar clientes.
RF04	O sistema deve permitir cadastrar e gerenciar endereços dos clientes.
RF05	O sistema deve permitir cadastrar um pedido para uma data e horário.
RF06	O sistema deve impedir o cadastro de pedido em data passada.
RF07	O sistema deve listar os próximos pedidos em ordem de data.
RF08	O usuário deve conseguir filtrar pedidos pelo calendário.
RF09	O usuário deve conseguir editar, cancelar e reagendar um pedido.

ID	Requisito
RF10	O usuário deve conseguir atualizar os status de pagamento e entrega.
RF11	O sistema deve mostrar relatórios e indicadores dos pedidos.
RF12	Quando configurado, o sistema deve criar ou atualizar o evento do pedido no Google Calendar.
RF13	Rotas de negócio devem exigir autenticação.

Requisitos não funcionais

ID	Requisito
RNF01	A interface deve funcionar em computador e possuir responsividade básica.
RNF02	O backend deve disponibilizar uma API REST.
RNF03	As senhas devem ser armazenadas de forma criptografada.
RNF04	A autenticação deve usar JWT em cookie HttpOnly.
RNF05	O banco utilizado deve ser MySQL 8.
RNF06	O sistema deve poder ser executado em containers Docker.
RNF07	A infraestrutura AWS deve ser versionada com Terraform.
RNF08	Backend e banco não devem ficar expostos diretamente à internet na AWS.
RNF09	Credenciais e senhas não devem ser salvas nos repositórios.
RNF10	O projeto deve possuir testes automatizados para os fluxos principais do backend.

6. Premissas

Para o projeto funcionar como planejado, consideramos que:

- a proprietária ou alguém da equipe fará o cadastro dos pedidos;
- os clientes não acessarão o sistema diretamente;
- a equipe terá acesso à internet e a um computador;
- as informações do pedido serão conferidas antes do cadastro;
- o administrador terá uma conta válida para acessar o sistema;
- a integração com agenda depende de uma conta de serviço válida do Google;
- o ambiente AWS Academy estará disponível durante testes e apresentações;
- as imagens do frontend e backend estarão publicadas no ECR antes do deploy;
- os dados usados em demonstrações não devem expor informações pessoais reais.

7. Restrições

- o projeto foi desenvolvido durante o período letivo e por uma equipe ainda em formação;
- a AWS Academy possui tempo de sessão e permissões limitadas;

- as credenciais da AWS Academy expiram e precisam ser renovadas;
- alguns serviços podem gerar custos, por isso o ambiente deve ser destruído após o uso;
- o Object Lock do S3 não pode ser consultado pelo perfil da Academy, então o Data Lake ficou desativado no Terraform;
- o sistema depende de internet para acessar a AWS e o Google Calendar;
- a integração com Google Calendar para de funcionar se a chave for revogada ou expirar;
- a versão atual utiliza MySQL em EC2, não Amazon RDS;

8. Artefatos relacionados às regras de negócio

Usuário principal

O usuário principal é a proprietária ou integrante da equipe da Doces com Amor. Ela precisa consultar rapidamente os pedidos, cadastrar informações e atualizar o andamento do trabalho.

Fluxo principal do pedido

```
graph TD;
    A[Login] --> B[Consulta dos próximos pedidos];
    B --> C[Seleção de uma data no calendário];
    C --> D[Cadastro do pedido];
    D --> E[Seleção do cliente e endereço];
    E --> F[Definição de data, horário, descrição e valor];
    F --> G[Salvamento no banco e tentativa de registro no Google Calendar];
    G --> H[Acompanhamento, edição, reagendamento ou cancelamento];
```

Regras de negócio principais

ID	Regra
RN01	Somente usuário autenticado pode acessar clientes, endereços, pedidos e relatórios.
RN02	No primeiro acesso, a troca da senha inicial é obrigatória.
RN03	Um pedido deve estar relacionado a um cliente e a um endereço.
RN04	Não é permitido cadastrar pedido em uma data anterior ao dia atual.
RN05	Para o dia atual, o horário deve ser posterior ao horário do cadastro.
RN06	Os próximos pedidos são exibidos do mais próximo para o mais distante.
RN07	Pedidos entregues continuam visíveis para facilitar a conferência.
RN08	Um pedido cancelado deixa de fazer parte do fluxo ativo, mas permanece no histórico e nos relatórios.

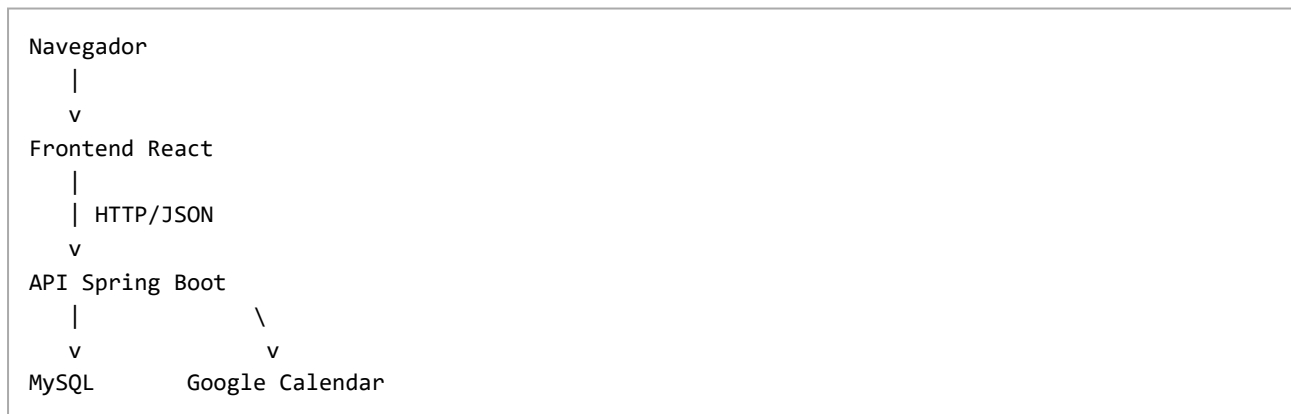
ID	Regra
RN09	O reagendamento deve registrar a nova data do pedido.
RN10	O status de pagamento cancelado não deve ser usado como atualização comum de um pedido ativo.
RN11	Cliente com pedido ativo não deve ser removido sem tratamento do vínculo.
RN12	Dados de relatório devem respeitar o período informado pelo usuário.

Jornada resumida

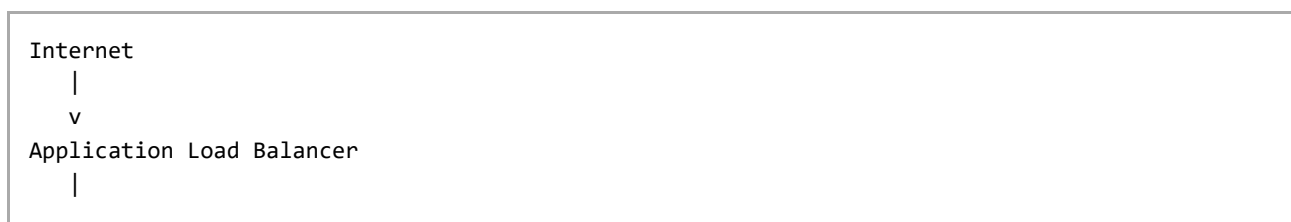
Momento	Ação da usuária	Resposta esperada do sistema
Entrada	Faz login.	Valida os dados e abre a área de pedidos.
Planejamento	Consulta calendário e próximos pedidos.	Mostra datas e pedidos em ordem.
Cadastro	Escolhe cliente, endereço, data e dados do pedido.	Valida e salva as informações.
Acompanhamento	Abre um pedido e consulta seus detalhes.	Exibe cliente, endereço, valor e status.
Atualização	Altera status, edita ou reagenda.	Salva a mudança e atualiza a listagem.
Análise	Abre o dashboard.	Exibe indicadores do período selecionado.

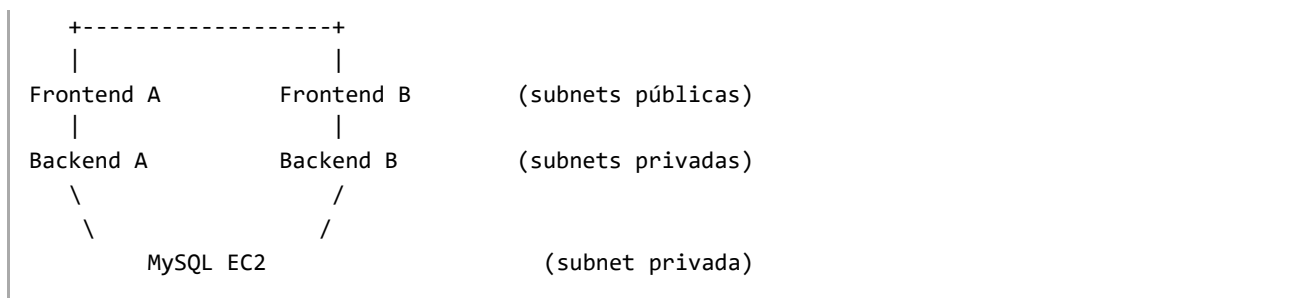
9. Artefatos relacionados às arquiteturas

Arquitetura da aplicação



Arquitetura de deploy na AWS

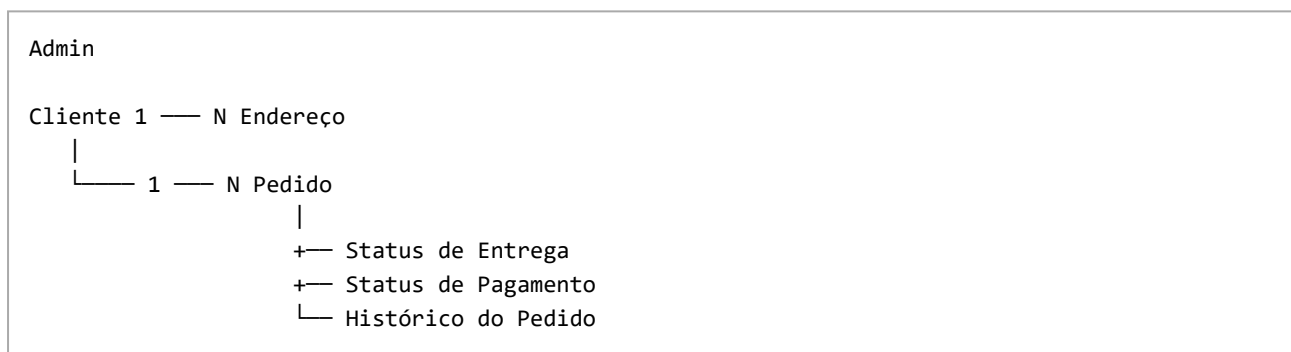




Outros componentes usados:

- Amazon ECR para armazenar as imagens Docker;
- NAT Gateway para saída das instâncias privadas;
- Security Groups para limitar a comunicação entre camadas;
- Secrets Manager para a credencial do Google Calendar;
- CloudWatch para dashboard, métricas e alarmes;
- SNS como destino dos alarmes;
- Terraform para criar e destruir o ambiente.

Organização do banco de dados



Entidades atuais do backend:

- Admin;
- Cliente;
- Endereco;
- Pedido;
- Entrega;
- Pagamento;
- HistoricoPedido.

Tecnologias

Camada	Tecnologias
Frontend	React 19, Vite, JavaScript e CSS
Backend	Java 17, Spring Boot, Spring Security, JPA e Swagger
Banco	MySQL 8

Camada	Tecnologias
Integração	Google Calendar API
Containers	Docker e Nginx
Cloud	AWS EC2, ALB, ECR, VPC, CloudWatch, SNS e Secrets Manager
Infraestrutura como código	Terraform
Automação	GitHub Actions

CI/CD atual

Após um merge na main do frontend ou backend, o GitHub Actions executa testes, faz o build da imagem e publica uma versão no ECR com a identificação do commit. O deploy da infraestrutura é feito separadamente pelo Terraform.

10. Product backlog

Prioridade	Item	Situação
Alta	Login e troca obrigatória de senha	Concluído
Alta	Proteção das rotas com JWT	Concluído
Alta	Cadastro e gestão de clientes	Concluído
Alta	Cadastro e gestão de endereços	Concluído
Alta	Listagem de pedidos e calendário	Concluído
Alta	Cadastro e detalhes do pedido	Concluído
Alta	Edição, cancelamento e reagendamento	Concluído
Alta	Atualização de pagamento e entrega	Concluído
Média	Dashboard e relatórios por período	Concluído
Média	Integração com Google Calendar	Concluído, depende de credencial válida
Média	Containers e publicação no ECR	Concluído
Média	Infraestrutura AWS com Terraform	Concluído
Média	CloudWatch e alarmes	Concluído em nível básico
Alta	Separar status do pedido, status da entrega, tipo de entrega, status do pagamento e meio de pagamento	Próxima versão
Alta	Não bloquear o pedido quando o Google Calendar falhar	Próxima versão
Média	Docker Compose para executar todo o projeto localmente	Próxima versão

Prioridade	Item	Situação
Média	Migrações de banco com Flyway	Futuro
Média	Centralização dos logs no CloudWatch	Futuro
Baixa	Integração real do Data Lake Bronze, Silver e Gold	Futuro
Baixa	Migração para RDS e ECS/Fargate	Futuro

11. Ferramentas de planejamento e organização

Ferramenta	Uso no projeto
Git e GitHub	Versionamento, branches, Pull Requests e histórico das entregas.
GitHub Actions	Testes, build e publicação das imagens Docker.
Trello	Organização do backlog em sprints com as colunas To Do, Doing e Done.
Figma	Referência visual para as telas.
Swagger	Consulta e teste dos endpoints do backend.
MySQL Workbench	Execução e conferência dos scripts do banco.
Terraform	Planejamento, criação e destruição da infraestrutura.
AWS Console	Conferência dos recursos durante testes e apresentações.
Reuniões e validações	Alinhamento das telas e do fluxo com a beneficiária.

O grupo trabalhou com branches por funcionalidade, revisão por Pull Request, commits em português e um quadro Kanban no Trello. O backlog foi separado por sprints e acompanhado pelas colunas To Do, Doing e Done.

12. Links do projeto no Git

- Documentação: <https://github.com/Grupo-9-2CCOA/Documentacao>
- Frontend: <https://github.com/Grupo-9-2CCOA/Frontend>
- Backend: <https://github.com/Grupo-9-2CCOA/Backend>
- Banco de Dados: <https://github.com/Grupo-9-2CCOA/Banco-de-Dados>
- Infraestrutura: <https://github.com/Grupo-9-2CCOA/infraestrutura>

13. Referências bibliográficas e técnicas

- AMAZON WEB SERVICES. **Documentação da AWS**. Disponível em: <https://docs.aws.amazon.com/>.
- DOCKER. **Docker Documentation**. Disponível em: <https://docs.docker.com/>.
- FOOD CONNECTION. **Tendências para o mercado de confeitaria**. Disponível em: <https://www.foodconnection.com.br/foodservice/5-tendencias-para-o-mercado-de-confeitaria/>.

- GOOGLE. **Google Calendar API**. Disponível em: <https://developers.google.com/calendar/api>.
- HASHICORP. **Terraform Documentation**. Disponível em: <https://developer.hashicorp.com/terraform/docs>.
- MYSQL. **MySQL 8.0 Reference Manual**. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>.
- REACT. **React Documentation**. Disponível em: <https://react.dev/>.
- SPRING. **Spring Boot Reference Documentation**. Disponível em: <https://docs.spring.io/spring-boot/>.